

Mingsong Liu

刘明松

Mathematics with Computer Science B.Sc. candidate

PERSONAL INFORMATION

GENDER Male

BORN December 2004

LOCATION Foshan, Guangdong, China

EMAIL liu24449@gtiit.edu.cn

PHONE +86 173 2270 5618

SKILLS

Python, Java, JavaScript, Game Boy (SM83) assembly language, xv6 kernel programming, C/C++

Node.js, npm, Vite, Git / GitHub, Linux, NumPy, SymPy, Matplotlib

AWARDS

Academic Excellence Award

GTIIT · 2024–2025 academic year

INTERESTS

Basketball, fitness

EDUCATION

Guangdong Technion – Israel Institute of Technology (GTIIT) Aug 2023 – Aug 2027

Shantou, China · B.Sc. in Mathematics with Computer Science

GPA 89.5 / 100 · Rank 15 (top 21.4%) · Taught entirely in English

Core coursework: Data Structures, Systems Programming, Computer Organization, Digital Systems and Computer Structure, Combinatorial Algorithms, Operating Systems, Numerical Analysis, Probability Theory, Combinatorics, Ordinary Differential Equations, Algebra, Infinitesimal Calculus, Introduction to Applied Mathematics

RESEARCH EXPERIENCE

Technion Summer Research Program Aug – Sep 2026

Technion – Israel Institute of Technology, Haifa

- Selected for the Technion Summer Research Program and studied quantum information theory under the supervision of Dr. Rajeev Gangwar.
- Analyzed entropy and divergence, the core measures of quantum information, and examined how they quantify the information carried by a quantum state and the distinguishability between two descriptions of a state.

PROJECT EXPERIENCE

Language of Thought (LoT) Sequence Complexity Framework Oct 2025 – Mar 2026

A minimal formal language for generating sequences, aimed at finding the smallest program that produces a given sequence — a computable analogue of Kolmogorov complexity.

- Implemented the LoT instruction set in Python, covering `play`, `move & play` and three loop constructs: fixed repetition, iterative mapping and iterative evaluation.
- Implemented direct generation of string sequences from a given program, and defined and implemented the computation of program space complexity.
- Implemented a dynamic-programming algorithm that computes the LoT complexity of a given string and returns the minimal program itself.
- Analyzed the time complexity of the algorithm and optimized it to polynomial time.

rgbds-live: Improvements to an Online Programming Interface May 2026 – Jun 2026

Tech: Node.js · JavaScript · npm · Vite · open-source contribution to rgbds-live, the browser IDE used in the Digital Systems course.

- Built and published an npm command-line tool that serves a local Game Boy assembly project over a local HTTP server and opens it in the web IDE, bypassing the 2 KB URL-hash project size limit while keeping all files on the user's own machine.
- Supported opening both remote URLs and local files directly from the terminal, streamlining the edit–compile–test workflow.
- Rewrote the assembly log parser to attach file and line metadata to compiler diagnostics, enabling one-click navigation from an error message to its source location.

Game Boy Assembly Game Development Dec 2024 – Jan 2025

- Developed a fully functional Klotski sliding-block puzzle game from scratch in Game Boy assembly, implementing pixel drawing, game logic, input handling and tile rendering.
- Studied the implementation patterns of early Game Boy titles and optimized instruction selection and memory usage for the hardware constraints of early computers.

ADDITIONAL PROJECTS

- **Restaurant fixed-menu system (Java)** — Designed and implemented category management, dependency management, menu generation and validation, combining double stacks, min-heap, AVL tree, Treap, Trie, hash table and graph structures.
- **Minimum vertex cover on undirected graphs** — Implemented an exact backtracking search accelerated by feasibility and optimality pruning, and a 2-approximation greedy algorithm to reduce time complexity.
- **Numerical methods (Python)** — Implemented Lagrange interpolation, least squares, RK4, Jacobi, Gauss-Seidel and SVD, and applied them to historical collision prediction, Moore's law trend analysis and Lorenz system simulation; used SVD rank to control image compression ratio.
- **xv6 operating system** — Implemented a user-level thread library, spinlocks and a concurrent memory allocator.